

DriveEase

ViewModels Layer

Request/response shapes and form validation

ASP.NET Core 9.0 MVC | Entity Framework Core | PostgreSQL (Supabase)

What ViewModels are

A ViewModel is a class shaped for a specific screen or request, sitting between the entity models and the views. It carries exactly the fields a form needs (no more), plus **DataAnnotation** validation attributes that both the browser (via jQuery validation) and the server (`ModelState.IsValid`) enforce. Using ViewModels keeps entities free of UI concerns and avoids over-posting.

Files & the ViewModels they contain

File	ViewModels
AuthViewModels.cs	RegisterViewModel, LoginViewModel
VehicleViewModels.cs	VehicleSearchFilterViewModel, RecommendationRequestViewModel, RecommendedVehicleViewModel
BookingViewModels.cs	BookingRequestViewModel, PriceBreakdownViewModel
PaymentViewModels.cs	MakePaymentViewModel
ReviewViewModels.cs	AddReviewViewModel
AdminViewModels.cs	AdminDashboardViewModel

1. RegisterViewModel

Captures sign-up input. Note the client-side password-match check via [Compare].

Property	Validation	Purpose
FullName	Required, 3-100 chars	User's name.
Email	Required, EmailAddress	Login email.
MobileNumber	Required, Phone	Contact/login mobile.
DrivingLicenseNumber	Required, max 30	Needed for self-drive.
Password	Required, min 6, Password	Account password.
ConfirmPassword	Required, Compare>Password	Must match Password.

2. LoginViewModel

Property	Validation	Purpose
EmailOrMobile	Required	Accepts either email or phone.
Password	Required, Password	Credential.

3. VehicleSearchFilterViewModel

All fields optional (nullable) - each supplied filter narrows the search.

Property	Type	Purpose
SearchLocation	string?	Filter by pickup location text.
VehicleType	VehicleType?	Bike / Hatchback / Sedan / SUV / Luxury.
MinPrice / MaxPrice	decimal?	Daily-rate range.
FuelType	FuelType?	Petrol / Diesel / Electric.
Transmission	TransmissionType?	Manual / Automatic.
SortBy	string?	'price_asc', 'price_desc', 'rating'.

4. RecommendationRequestViewModel

Input to the recommendation scoring engine, with range validation.

Property	Validation	Purpose
Passengers	Range 1-15	Group size to fit.
Budget	Range 100-1,000,000	Spend limit (Rs).
PreferredType	optional	Desired vehicle category.
DurationDays	Range 1-30	Rental length in days.

5. RecommendedVehicleViewModel (output)

Returned by the engine - a scored suggestion, not user input.

Property	Type	Purpose
Vehicle	Vehicle	The suggested vehicle.
Score	double	Ranking score (out of ~100).
Reason	string	Human-readable justification (bullet joined).
EstimatedTotalCost	decimal	DailyRate x DurationDays.

6. BookingRequestViewModel

Input to create a booking. Defaults set sensible pickup/return times.

Property	Validation / Default	Purpose
VehicleId, UserId	Required	Who is booking what.
RentalType	default Daily	Hourly or Daily pricing.
PickupDateTime	Required, default now+2h	Start of rental.
ReturnDateTime	Required, default now+1d	End of rental.
PickupLocation	Required	Pickup point.
PromoCode	optional	Discount code to apply.

7. PriceBreakdownViewModel (output)

Computed by BookingService.CalculatePriceAsync and shown to the user before payment.

Property	Purpose
RentalType, Duration, UnitRate	How the base was computed (rate x units).
BasePrice	Subtotal after any surge multiplier.
DynamicMultiplier	Surge factor applied (e.g. 1.15 on weekends).
AppliedPromoCode	Promo code that was accepted, if any.
DiscountAmount	Value subtracted by the promo.
TotalAmount	Final payable = BasePrice - DiscountAmount.

8. MakePaymentViewModel

Property	Purpose
BookingId	Which pending booking is being paid.
PaymentMethod	Card / UPI / NetBanking / Wallet.
Amount	Payable amount (must equal booking TotalAmount).

9. AddReviewViewModel

Property	Validation	Purpose
BookingId	Required	Booking being reviewed.
Rating	Range 1-5	Star rating.
Comment	Required, 5-1000 chars	Review text.

10. AdminDashboardViewModel (output)

Aggregated stats produced by AdminService for the admin dashboard.

Property	Purpose
TotalUsers	Count of customers.
TotalVehicles / Available / UnderMaintenance	Fleet counts by status.
TotalBookings / Active / Completed / Cancelled / Pending	Booking counts by status.
TotalRevenue	Sum of successful payments.
RecentBookings	5 most recent bookings for a quick view.

Why ViewModels instead of entities

Binding a form directly to an entity risks **over-posting** (a user could set fields like Role or IsActive that were never on the form) and mixes validation rules into the persistence model. ViewModels expose only the intended fields and hold the UI validation, keeping entities clean.